

CENG444 - Language Processors

2025-2026 Spring

Project II**Syntactic analysis with Bison**

Problem

In this assignment, you are required to write a C++ program that performs syntactic conformance test for an experimental language called as *rowset manipulation language (RML)*. This experimental language will also be the basis of the forthcoming assignments on semantic analysis and code generation. The problem particular to this assignment is implementation of a syntax checker which requires definitions for scanner and parser, and implementation of a command line tool that reports syntactic conformity of a given RML module.

RML

An RML module consists of a series of “select” statements. A valid RML statement conforms to a set of syntactic rules that are mostly similar to the rules of the conventional expressions we are familiar with. The type system of RML is based on primitive types, which are string, number, and Boolean and rowset. RML does not have flow control structures like loops, decisions, and compound statements. However, implementation of a “select” statement involves implicit loops and conditional execution components without explicit constructs that we are familiar from imperative languages.

RML manipulates CSV files each having a header row and zero or more data rows. You do not need to worry about processing details of a CSV file because a supplemental library will be provided to help you focus on essence of language processing. **Additionally, this particular assignment is solely about syntax checking, you will not need to use and apply CSV processing techniques.**

Types

Since the `rowset` type can be reduced to other basic types at runtime, RML can be said to be a dynamically typed language. This does not mean that compile time checking (static type checking) is completely deferred. There is no explicit variable declaration construct in RML to define a variable. Instead, variable types are received from the CSV files. Any column defined in a CSV file becomes a variable that may receive partially or full qualified references. Once a variable is declared, its type cannot be changed.

Below is the list of the types allowed in RML.

Type	Description
number	Type to represent IEEE 754 double precision floating point value.
boolean	Type to represent boolean values.

Type	Description
string	String of characters.
rowset	An object that encapsulates rows each of which is composed of columns. When a rowset has a single row and a single column, it can be reduced to number, Boolean, or string depending on the column type.

Select Statements

“Select statements” is the only kind of statements that can be found in a valid RML program. An RML program must contain at least one select statement. A “select statement” is formed by placing a semicolon after a “select operation”.

The “select statement” is the major element of an RML module. Each select statement contains a select operation, which is a ternary operator. The operands are named as “column list”, “product space”, and “condition”.

Column List: A column list is an expression that may result in multiple values by using comma operator.

Product Space: Syntactically, a product space is a comma separated list of rowsets implying the cross product of the spaces to form the final rowset that will be iterated over for calculation. A rowset appearing in the list is called as a **space**. A space can be declared in two ways: External or calculated. An external space is a single identifier that references to a csv file. In such a case, the file is located by using the filename synthesized as “<identifier>.csv”. A calculated space is a select statement enclosed in parentheses. Any space may have an alias that makes it possible to arrange unambiguous rowset references from within the expressions.

Condition: A Boolean value evaluating expression that is used to qualify each row in the product space. A condition must not involve top level comma operator which leads production of more than one value.

Example:

```
select A.name, A.surname, B.grade
  from (select P.id, P.grade from student P where P.typ=="external") A, coursegrading B
  where B.studentid==A.id && B.grade>=70;
select max(grade) from coursegrading where coursecode=="CENG444";
select A.name, A.surname from student A, coursegrading B
  where B.studentid==A.id && B.grade==(select max(grade) from coursegrading where true);
```

In this example, B.grade is a fully qualified variable, and the “P” in “student P” is an alias. You can spot more than one fully qualified variables and aliases.

Expressions

The form of the expressions in RML is mostly conventional. This section defines the entities that can be used to construct a valid RML expression.

In this assignment, you are required to check only for syntactic conformance. Do not get into semantic details like type checking, type evaluation, precedence, associativity, constant folding, parameter checking, declarative integrity, and similar.

- Operators

RML has a rich set of operators with various number of operands and associativities. You are accustomed to many of them from your earlier experience. See **Error! Reference source not found.**

- Parentheses

An expression can be enclosed in parentheses to change / ensure evaluation order. See the precedence and associativity of the operators to understand the default evaluation order.

Additionally, any rowset, which is a first class citizen in RML, must be a “select” enclosed in parentheses.

- String Literals

A string is in the form of regular and exceptional characters enclosed in double quotes. Exceptional characters are double quote(“), and back-slash(\), and new-line(character code 10).

Exceptional Character	Encoding in string
double quote(“)	\”
back-slash(\)	\\
new-line	\n

A character, regardless if it is exceptional or regular, can be coded with help of byte coding. Any byte except zero can be encoded in a string of characters by using the hexadecimal form, which is \xhh, where h is a hexadecimal digit. Capital and small letters can be used in hexadecimal digit specification.

- Numeric Literals

The numeric literals can be composed of the valid combinations of whole part (W), decimal separator (.), and fractional part (F). Whole part can be either zero, or a sequence of digits starting with a non-zero digit. Fractional part is a nonempty sequence of digits, not ending with zero. Following are valid examples of W, “.”, and F combinations:

W	.	F	Example
1	1	1	1.25
1	1	1	0.25432
0	1	1	.25
0	1	1	.0134
1	0	0	123
1	0	0	0

A number can have an optional exponential part prefixed by ‘e’ or ‘E’. When supplied, an optional

sign that can be either '+' or '-' may follow. Then, a sequence of digits must be given. The first digit must be non-zero.

- Boolean Literals

true and **false** are the boolean literals.

- Identifiers

An identifier starts with a letter or an underscore. The remaining characters in an identifier can be letters, underscores, and decimal digits. RML identifiers are case sensitive so `var` and `Var` are two distinct variables.

- Function Calls

A function call is formed as <identifier> followed an expression list enclosed by parentheses. The list of expressions is skipped for the functions having no formal parameters.

White Spaces

The tokens found in a valid RML code must be separated by at least one white space character, or special character such those that can be found in operators. The developer is free to use as many white spaces as required to improve readability of the code.

RML Program

A valid RML program is a sequential list of statements that satisfy following conditions:

1. Statements must be syntactically correct.

This will be ensured by your program.

2. Statements must be free of semantic errors.

This will be ensured by your forthcoming assignment besides other requirements. You need not to worry about semantic checks in this experiment.

Processor Output

The processor you develop (**project02**) will accept the name of the source file that will be syntactically analyzed from the command line. The command must contain only one parameter. No parameter or more than one parameter cases must be reported as error. The processor must create an output file (**project02.txt**) containing 9 lines with collected frequencies in case it recognizes the input as a syntactically valid program. The strict ordering of the lines is given below.

<Total number of statements>

<Total number of selects>

<Total number of spaces>

<Total number of aliases>

<Total number of numeric constants>

<Total number of string constants>

<Total number of variable references>

<Total number of fully qualified variable references>

<Total number of function calls>

In case the input file is found syntactically non-compliant, the output file (**project02.txt**) will contain a single line as follows.

Line <line number>: Syntax error.

Regulations & Hints

- **Implementation:** You should use Flex, Bison with C++ to develop your program. Make sure that your program will accept the name of the input file. In case the program is run without a parameter or with more than one parameter your program must terminate immediately with appropriate error message sent to the standard output.
- **Evaluation:** The evaluation will be based on correct diagnosis of 5 different input files, two points for each.
- **Submission:** You need to submit all relevant files you have implemented (.cpp, .h, .l, *.y, etc.) as well as a README file with instructions on how to run, in a single .zip file named 1234567.zip with your studentID.

Following notes are for more clarity to address possible concerns and / or questions.

- Use Flex and Bison to generate C++ not C file. C implementation will not be accepted.
- Never modify the automatically generated files generated by Flex and Bison. You may prefer consulting the recitation material and the sample project on syntactic analysis.
- Describe precisely the steps to build and run your program in README.txt file, which is essential part of your submission. Provide any configuration items such as scripts, configuration files that will be necessary.

Appendix 1: Operators

Below are the operators that can be used in RML expressions. For each operator, form (coding syntax), possible applications on types, type of the result, associativity, and precedence rules are given.

OP	Short Name	Form	Application	Result Type	Assoc.	Prec.
Select From where	Selection	Ternary	(select <expression> from <product-space> where <expression>)	rowset	N/A	N/A
,	Comma	Binary		Multi	LR	0
?:	Conditional	Ternary	<boolean>?<type1>:<type2>	<type1>		1
&&	Boolean and	Binary, infix	<boolean> && <boolean>	<boolean>	LR	2
	Boolean or	Binary, infix	<boolean> <boolean>	<boolean>	LR	2
==	Equal	Binary, infix	<type1> == <type2>	<boolean>	LR	3
!=	Not equal	Binary, infix	<type1> != <type2>	<boolean>	LR	3
<	Less than	Binary, infix	<type1> < <type2>	<boolean>	LR	3
>	Greater than	Binary, infix	<type1> > <type2>	<boolean>	LR	3
<=	Less than or equal	Binary, infix	<type1> <= <type2>	<boolean>	LR	3
>=	Greater than or equal	Binary, infix	<type1> >= <type2>	<boolean>	LR	3
+	Add	Binary, infix	<number> + <number> <number> + <rowset> <rowset> + <number>	<number> <number> <number>	LR	4
+	Concatenate	Binary, infix	<string> + <string> <string> + <number> <number> + <string> <string> + <rowset> <rowset> + <string>	<string> <string> <string> <string> <string>	LR	4
-	Subtract	Binary, infix	<number> - <number>	<number>	LR	4
*	Multiply	Binary, infix	<number> * <number>	<number>	LR	5
/	Divide	Binary, infix	<number> / <number>	<number>	LR	5
-	Negate	Unary, prefix	-<number>	<number>	RL	6
!	Boolean negate	Unary, prefix	!<boolean>	<boolean>	RL	6
()	Call function	Binary	id(<expression>)	Return type of function	LR	7